

Github Workshop

How to utilize the software
'Git' on the platform 'Github'

TABLE OF CONTENTS

01

Set-up environment

Setting up your environment w/
Git and Github

02

Staging and Unstaging

Staging and unstaging changes
in a repository

03

Pushing and pulling

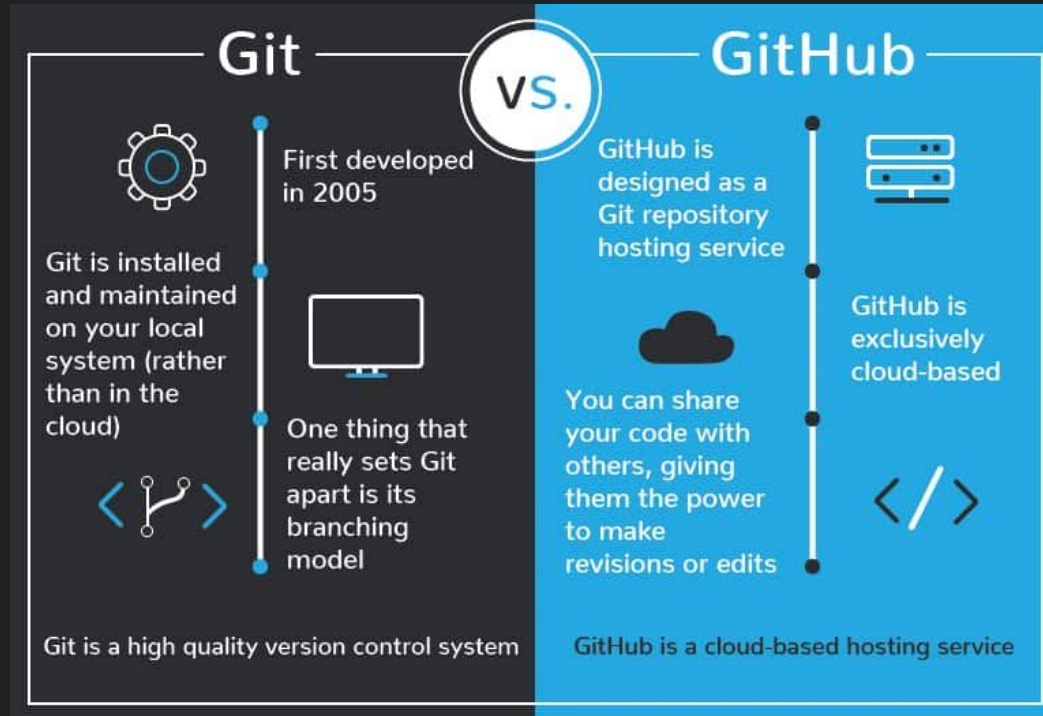
Pushing and pulling commits

04

Separate Branches

Creating separate branches

Git is **not** the same thing as Github





01

Set-up your
environment



Download Git

Not sure if you have Git downloaded? Open up a new terminal in Visual Studio, IntelliJ, RStudio, etc. or a new terminal window in general.

In your terminal, type in the command 'git' on a new line. Something like the image to the right should pop up if git is properly installed.

If nothing pops up or an error is displayed, then download git from <https://git-scm.com/>

```
jay@Jays-MacBook-Air ~ % git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
          [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
          [--super-prefix=<path>] [--config-env=<name>=<envvar>]
          <command> [<args>]
```

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)

clone	Clone a repository into a new directory
init	Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)

add	Add file contents to the index
mv	Move or rename a file, a directory, or a symlink
restore	Restore working tree files
rm	Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)

bisect	Use binary search to find the commit that introduced a bug
diff	Show changes between commits, commit and working tree, etc
grep	Print lines matching a pattern
log	Show commit logs
show	Show various types of objects
status	Show the working tree status

grow, mark and tweak your common history

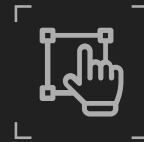
branch	List, create, or delete branches
commit	Record changes to the repository
merge	Join two or more development histories together
rebase	Reapply commits on top of another base tip
reset	Reset current HEAD to the specified state

Set-up Github



Go to <https://github.com/> and create a new account if you have not done so. Then, go to your dashboard

Hit either  or the plus sign to create a new repository.





Dashboard

Q Type **Z** to search



Top repositories

New

Find a repository...

- Jay-Madame/CS2_Module2_Classwork
- Jay-Madame/ModuleLab2Yahtzee
- Jay-Madame/PEMMDASQuizzerJC
- Jay-Madame/SD_LabOne
- Jay-Madame/testingGit
- Jay-Madame/exit-ticket-jan14
- Jay-Madame/hello-world-SD25

Show more

Join GitHub Education!

GitHub Education opens doors to new skills, tools, and a collaborative community eager to drive innovation. Join us and build a foundation for your future in technology.

Free and discounted services for teachers and students.



Copilot
Turn natural language prompts into coding suggestions



Heroku
Build, run, and operate applications entirely in the cloud.



Microsoft Azure
Access to Microsoft Azure cloud services and learning resources

Join GitHub Education

Ask Copilot



Latest nodejs/node release

Open issues in facebook/react

Home

Filter

Trending repositories · [See more](#)

GitHub Copilot



Now available for free

Up to 50 chats and 2,000 code completions per month, access to Claude 3.5 Sonnet, and more.

Open Copilot

Latest changes

- 4 days ago
What's New in Mobile, January Update
- 5 days ago
GitHub Models introduces a new prompt editor (GA)
- 5 days ago
Copilot in GitHub.com now supports Content Exclusions (Preview)
- 5 days ago
Legacy Docker Registry closing down
[View changelog](#) →

Explore repositories

scioV / scioV



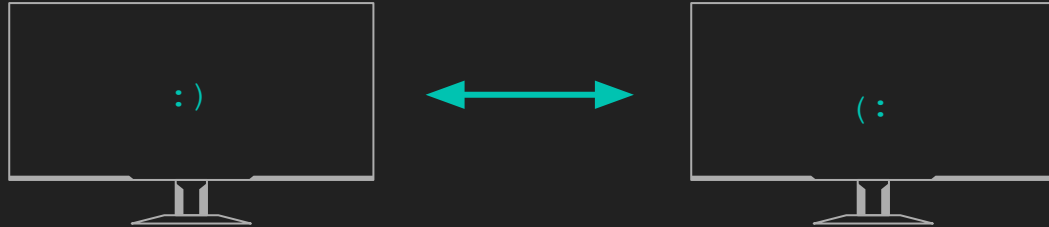
02

Staging/unstaging changes in a Git Repo



What is a “repo?”

A repository is a place where you can store shared code with friends, co-workers, classmates and even complete strangers. As a result, repositories help individuals and teams write better code faster.



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner *

 Jay-Madame ▾

Repository name *

/

Great repository names are short and memorable. Need inspiration? How about [urban-octo-carnival](#) ?

Description (optional)

☒  Public

Anyone on the internet can see this repository. You choose who can commit.

☐  Private

You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: **None** ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: **None** ▾

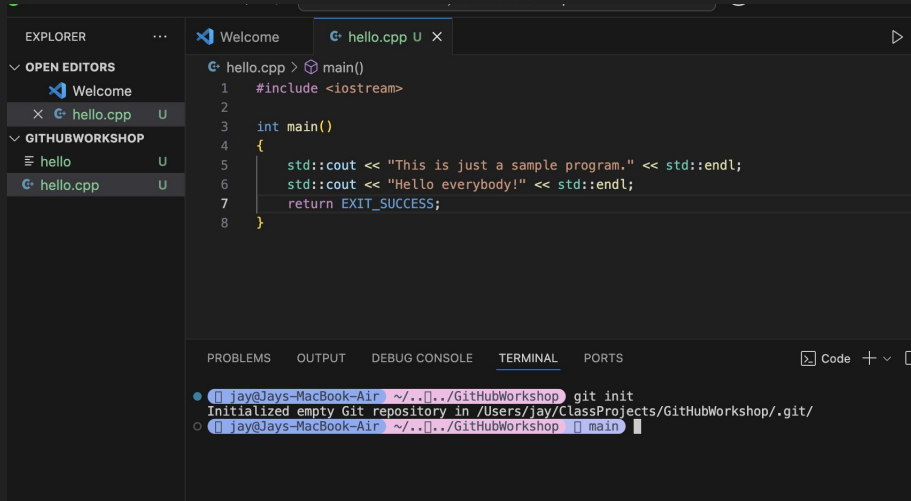
A license tells others what they can and can't do with your code. [Learn more about licenses](#).

All you have to do on this tab is really just type in your repository name, and then hit “Create repository on the bottom! The other stuff is not something to worry about as an underclassmen just yet.

How to Stage Changes

Let's go ahead and open your IDE of your choice. For this demonstration, I'm using VS Code.

As you can see, I have a terminal open on VSCode. In this terminal, I'm going to type "git init"

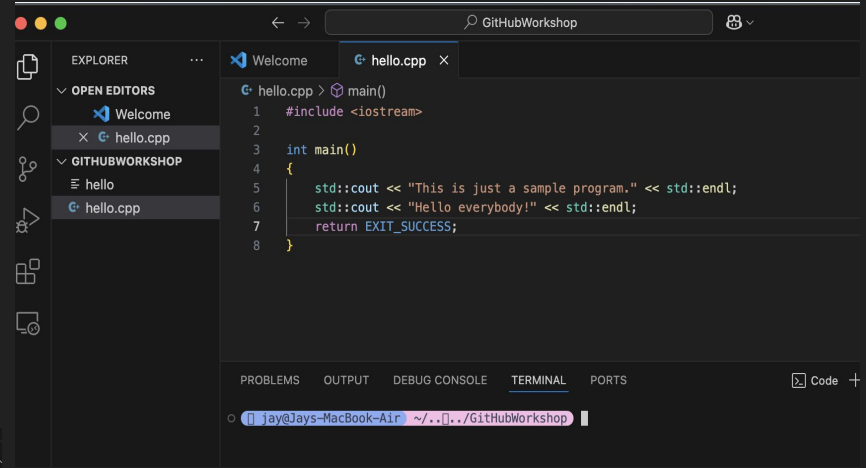


This screenshot shows the VS Code interface. The Explorer sidebar on the left shows the file structure with 'hello.cpp' marked with a green 'U' icon, indicating it is untracked. The main editor area displays the code for 'hello.cpp'. At the bottom, the Terminal panel is active, showing the command prompt 'jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop' and the command 'git init' which has been executed, resulting in the output 'Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/'.

```
EXPLORER
  OPEN EDITORS
    Welcome
    X hello.cpp U
  GITHUBWORKSHOP
    hello U
    hello.cpp U

hello.cpp > main()
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "This is just a sample program." << std::endl;
6     std::cout << "Hello everybody!" << std::endl;
7     return EXIT_SUCCESS;
8 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop git init
Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/
jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop main
```



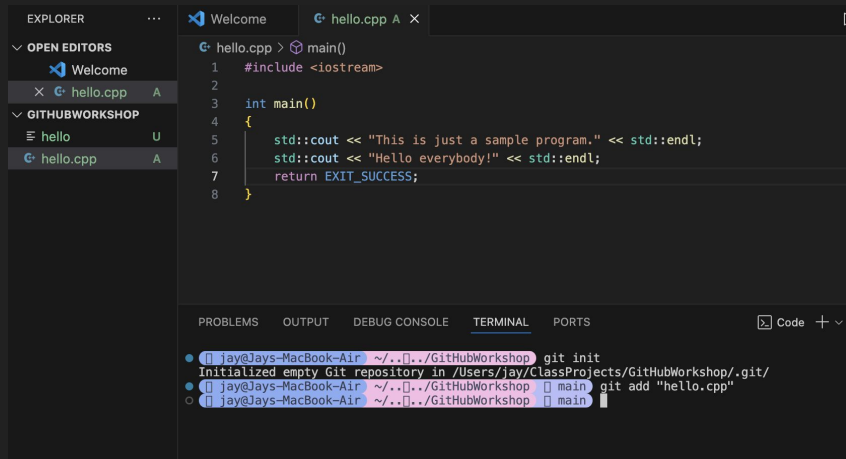
This screenshot shows the VS Code interface. The Explorer sidebar on the left shows the file structure with 'hello.cpp' marked with a green 'U' icon, indicating it is untracked. The main editor area displays the code for 'hello.cpp'. At the bottom, the Terminal panel is active, showing the command prompt 'jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop' and the command 'git init' which has been executed, resulting in the output 'Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/'.

```
EXPLORER
  OPEN EDITORS
    Welcome
    X hello.cpp U
  GITHUBWORKSHOP
    hello U
    hello.cpp U

hello.cpp > main()
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "This is just a sample program." << std::endl;
6     std::cout << "Hello everybody!" << std::endl;
7     return EXIT_SUCCESS;
8 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop git init
Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/
jay@Jays-MacBook-Air: ~/.././/GitHubWorkshop main
```

You may notice, that the files turn green with a "U" symbol next to them. This means that the files are untracked. The command "git init" only initializes a repo, but does not add any files you have made in your current directory.



EXPLORER

OPEN EDITORS

- Welcome
- hello.cpp

GITHUBWORKSHOP

- hello
- hello.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "This is just a sample program." << std::endl;
6     std::cout << "Hello everybody!" << std::endl;
7     return EXIT_SUCCESS;
8 }
```

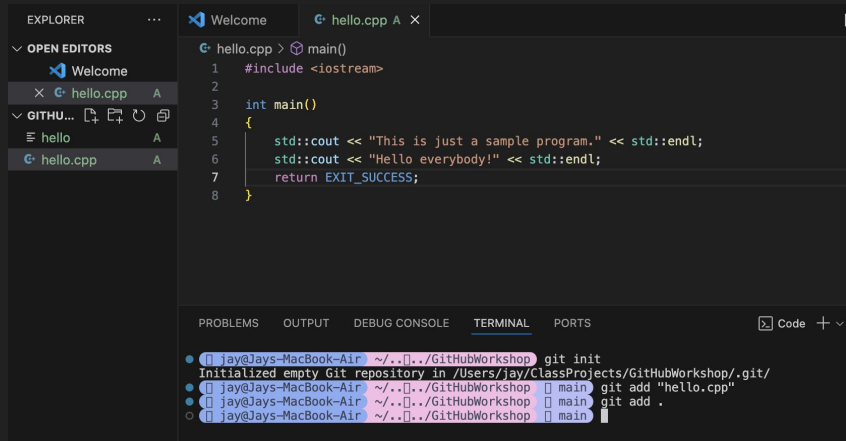
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ git init
Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ main git add "hello.cpp"
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ main
```

To add files to your repo to stage for commits, you can choose two options:

1. 'git add [filename]'

This is how to manually add files one by one.



EXPLORER

OPEN EDITORS

- Welcome
- hello.cpp

GITHU...

- hello
- hello.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "This is just a sample program." << std::endl;
6     std::cout << "Hello everybody!" << std::endl;
7     return EXIT_SUCCESS;
8 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ git init
Initialized empty Git repository in /Users/jay/ClassProjects/GitHubWorkshop/.git/
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ main git add "hello.cpp"
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ main git add .
jay@Jays-MacBook-Air: ~/.../GitHubWorkshop$ main
```

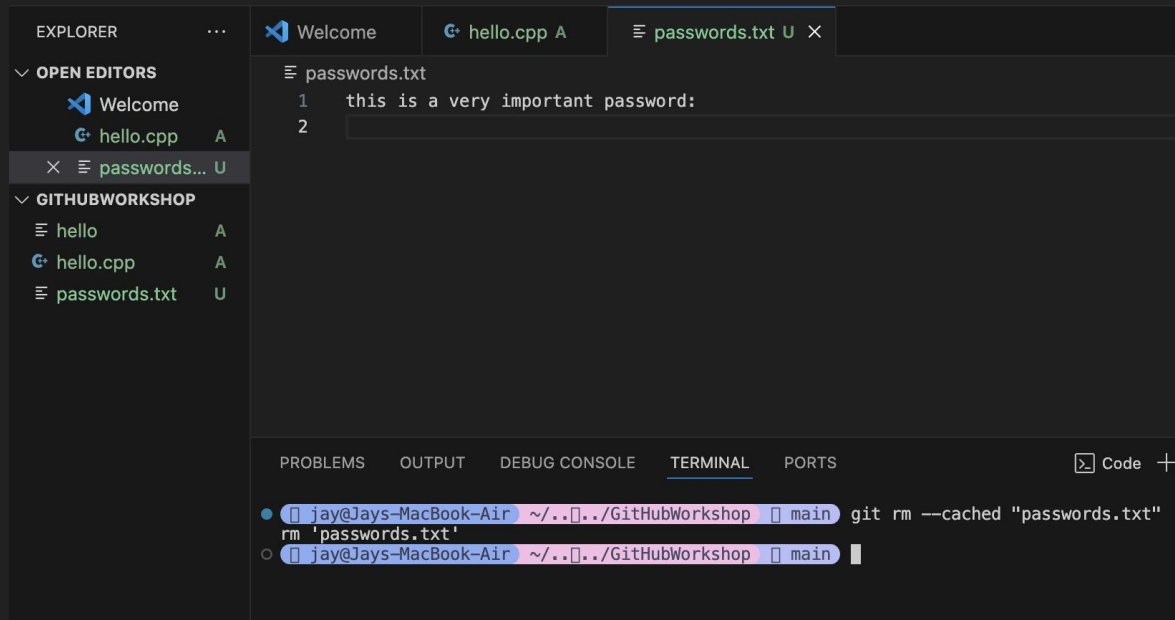
2. 'git add .'

This is adds all the files in your directory to the repo

How to Unstage Changes

There are multiple different ways to unstage a change by removing a file from your repository. Let's say we create "passwords.txt" that contain all of our important passwords, and we accidentally typed in 'git add .'

Now, "passwords.txt" is apart of the repository, but we haven't committed this change just yet. So in order to remove "passwords.txt" from the repo, we can type in 'git rm --cached "passwords.txt"'



```
EXPLORER  ...  
v OPEN EDITORS  
  Welcome  
  hello.cpp A  
  passwords.txt U  
v GITHUBWORKSHOP  
  hello A  
  hello.cpp A  
  passwords.txt U  
  
Welcome  
hello.cpp A  
passwords.txt U  
passwords.txt  
1  this is a very important password:  
2  
  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● jay@Jays-MacBook-Air ~/..../GitHubWorkshop main git rm --cached "passwords.txt"  
  rm 'passwords.txt'  
○ jay@Jays-MacBook-Air ~/..../GitHubWorkshop main
```

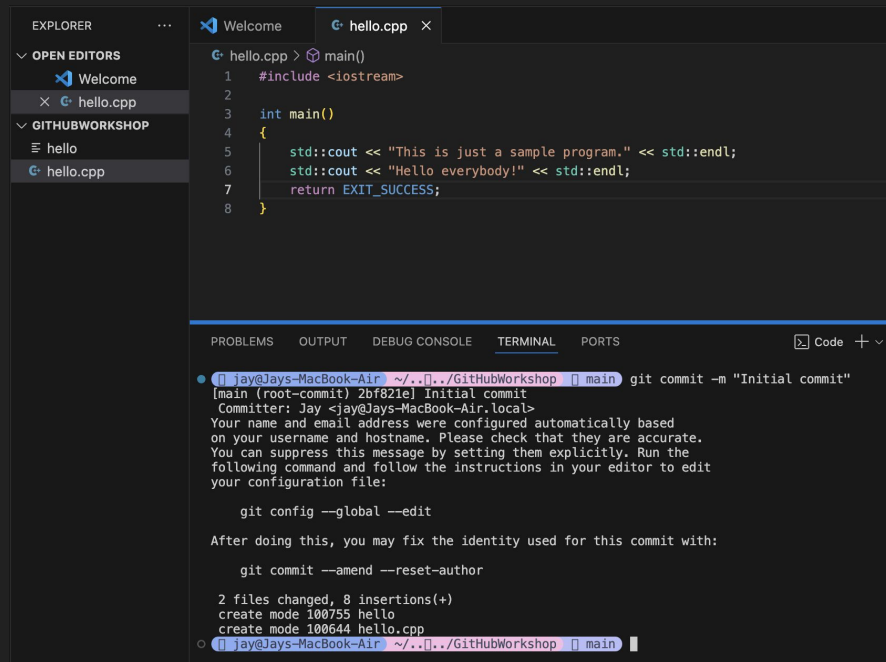
Note: It's important to add '--cached' between rm and your filename to avoid accidentally deleting the file off your local machine

Committing those changes

To save our first round of changes, we'd type 'git commit -m "Initial commit"' (or something along those lines).

It doesn't matter if you use single quotes ('), or double quotes (""), but no matter what, the end of the message is marked by the second quote.

If you accidentally messed up your commit, you can hit command + c on macOS, or use ctrl + c on windows. This will exit you out of the command line, and you can restart from there.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'GITHUBWORKSHOP' with two files: 'hello' and 'hello.cpp'. The 'hello.cpp' file is open in the editor, showing the following code:

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "This is just a sample program." << std::endl;
6     std::cout << "Hello everybody!" << std::endl;
7     return EXIT_SUCCESS;
8 }
```

At the bottom, the Terminal window is open, showing the output of the 'git commit -m "Initial commit"' command. The output includes the commit hash '2bf821e', the commit message 'Initial commit', and the author 'Jay <jay@Jays-MacBook-Air.local>'. It also shows the number of files changed and the number of insertions.

```
jay@Jays-MacBook-Air: ~/.[...]/GitHubWorkshop [main] git commit -m "Initial commit"
[main (root-commit) 2bf821e] Initial commit
Committer: Jay <jay@Jays-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

2 files changed, 8 insertions(+)
create mode 100755 hello
create mode 100644 hello.cpp
jay@Jays-MacBook-Air: ~/.[...]/GitHubWorkshop [main]
```

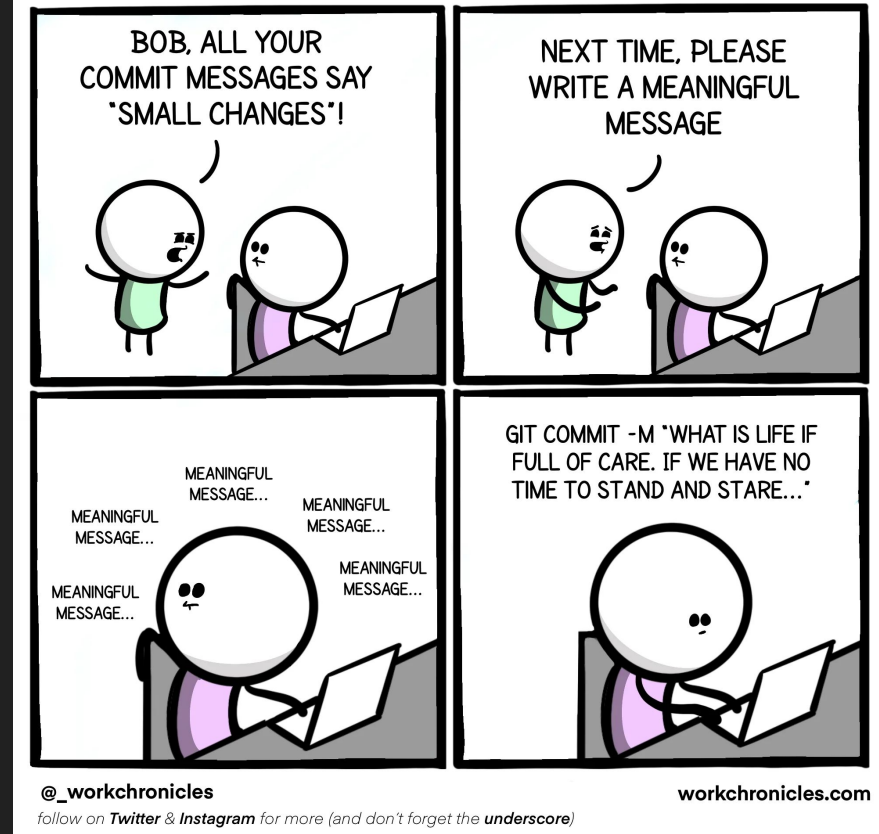
A commit message is helpful to other collaborators on your project.

Good commit messages:

- Short
- Meaningful
- Reflect the changes made

Bad commit messages:

- One word
- Not relating to changes or the project
- Include profanity or slang
- Way too elaborate



Examples of commit messages

Good:

“fix: display error in register.java”

“feat: implemented makingChange function”

“chore: updated dependencies”

[\(examples of conventional commits\)](#)

Bad:

“i’m going to kms why does this not work”

“Changed line in line 72 to be std::cout<<Hello\n instead of std::cout<<Hello<<std::endl”

“second commit”

“2”



03

Pushing and pulling



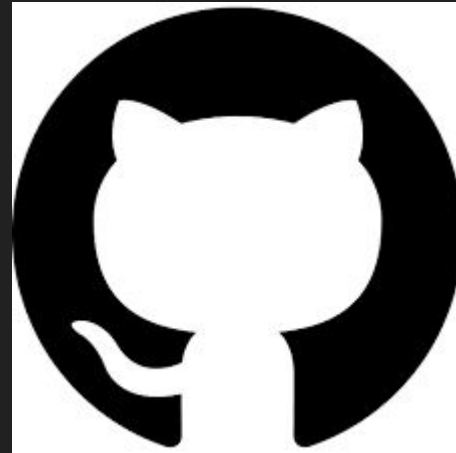
Local

The repo on **your** computer.



Remote

A remote is a repository made available by Git hosting platforms like Github.



Most git projects have a single remote that is used as a 'source of truth.' Collaborators (i.e. - your teammates) on the project push and pull to and from this remote. Because this is the source of truth, it is commonly named 'origin.'

You might think that your local repository should be considered origin, since you're pushing from your repository into Github. However, git is a collaborative tool. If you were to follow this logic, then all of your collaborators on a project would have their own source of truth.

Going back to Github, you may have a confusing view like so (if you are unused to Github's window when you create a new repo):

Quick setup — if you've done this kind of thing before



Set up in Desktop

or

HTTPS

SSH

`https://github.com/Jay-Madame/githubWorkshop.git`



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# githubWorkshop" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Jay-Madame/githubWorkshop.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/Jay-Madame/githubWorkshop.git
git branch -M main
git push -u origin main
```



For this tutorial, we'll be following these lines right here.

...or push an existing repository from the command line

```
git remote add origin https://github.com/Jay-Madame/githubWorkshop.git
git branch -M main
git push -u origin main
```



For macOS/Linux users, you may have to input these lines instead into your terminal's command line

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

 Code + -   ... ^ X

```
• jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git remote add origin https://github.com/Jay-Madame/g
ithubWorkshop.git
• jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.00 KiB | 2.00 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Jay-Madame/githubWorkshop.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
○ jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main]
```


Push

Push

```
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main]
```

A push will push everything off your computer (the local repo), onto the remote repo. A push will get your most recent commit, and push those changes onto the main branch.

Note** - the main branch may be known as a master branch on different computers.

Pull

Pull

```
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git pull
Already up to date.
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main]
```

A pull will get any changes made to the main branch, and will bring those changes to your computer (the local repo).

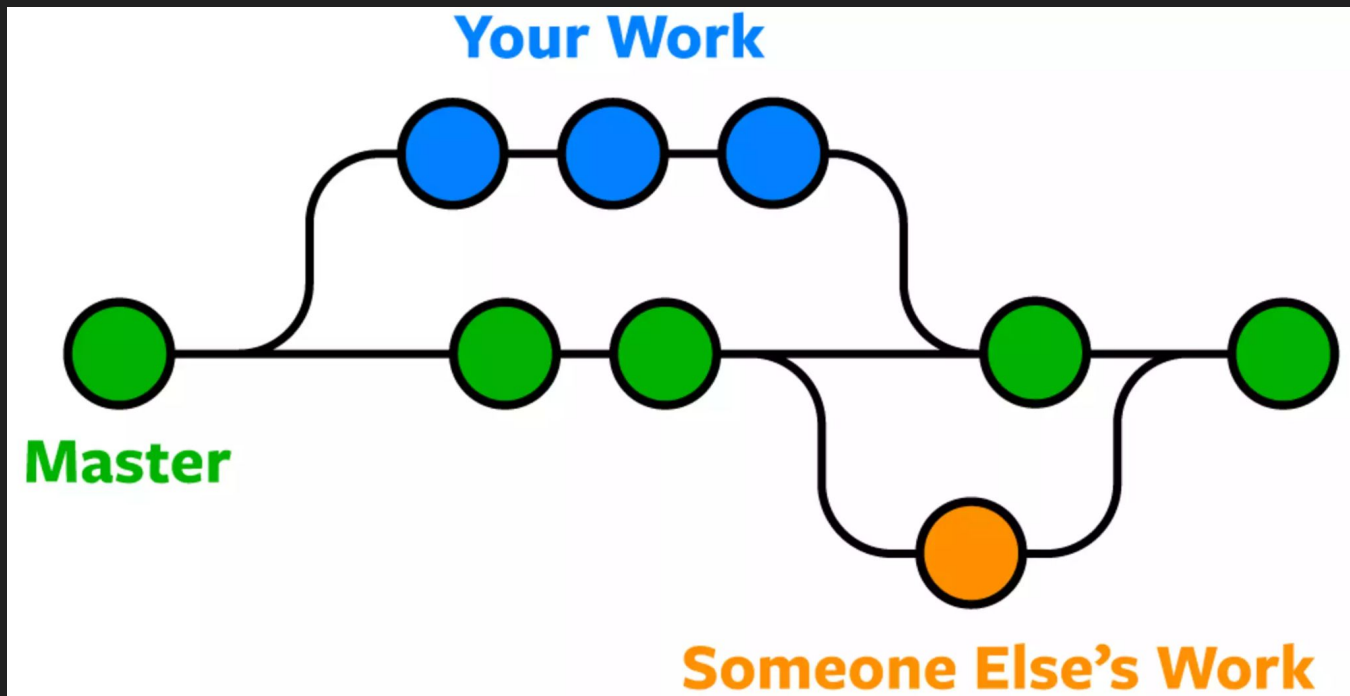
Make sure...

As a programmer, you want to do constant pushes and pulls, to make sure your work is on track with the master branch off the remote repository. You could type in `git status` to see if you need to commit changes, and if you're on track with the main/master branch

```
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Why you push/pull:





04

Separate Branches



Separate branches

In Github, if you don't want to work on the main branch, you can create a new branch. To create a new branch, type in the command line “git checkout -b newBranch”

Obviously, you can rename newBranch to whatever you may like. Just for simplification of this tutorial, we'll call it newBranch

```
jay@Jays-MacBook-Air ~/../../GitHubWorkshop [main] git checkout -b newBranch  
Switched to a new branch 'newBranch'
```

Checkout will switch branch, and -b stands for creating a new branch. You could also use git switch [branchname] to switch between branches

newBranch

```
1  
3  int main()  
4  {  
5      std::cout << "This is just a sample program." << std::endl;  
6      std::cout << "Hello everybody!" << std::endl;  
7      std::cout << "I am adding a line :)" << std::endl;  
8      return EXIT_SUCCESS;  
9  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

> jay@Jays-MacBook-Air ~/../../GitHubWorkshop newBranch

main

```
3  int main()  
4  {  
5      std::cout << "This is just a sample program." << std::endl;  
6      std::cout << "Hello everybody!" << std::endl;  
7      std::cout << "I am adding a line" << std::endl;  
8      return EXIT_SUCCESS;  
9  }
```

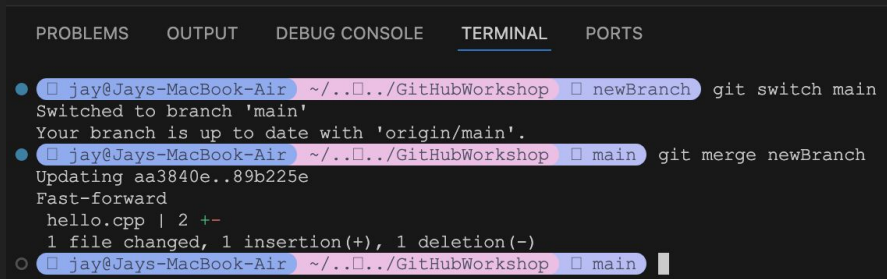
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

o jay@Jays-MacBook-Air ~/../../GitHubWorkshop main

Option One - Merge

As you can see, merge will add whatever is in the specified branch and add it to the current branch.

```
3  ∨ int main()
4  {
5      std::cout << "This is just a sample program." << std::endl;
6      std::cout << "Hello everybody!" << std::endl;
7      std::cout << "I am adding a line :)" << std::endl;
8      return EXIT_SUCCESS;
9  }
```



The screenshot shows a terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• [jay@Jays-MacBook-Air ~/.../GitHubWorkshop] newBranch git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
• [jay@Jays-MacBook-Air ~/.../GitHubWorkshop] main git merge newBranch
Updating aa3840e..89b225e
Fast-forward
 hello.cpp | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
○ [jay@Jays-MacBook-Air ~/.../GitHubWorkshop] main
```

Option Two - Rebase

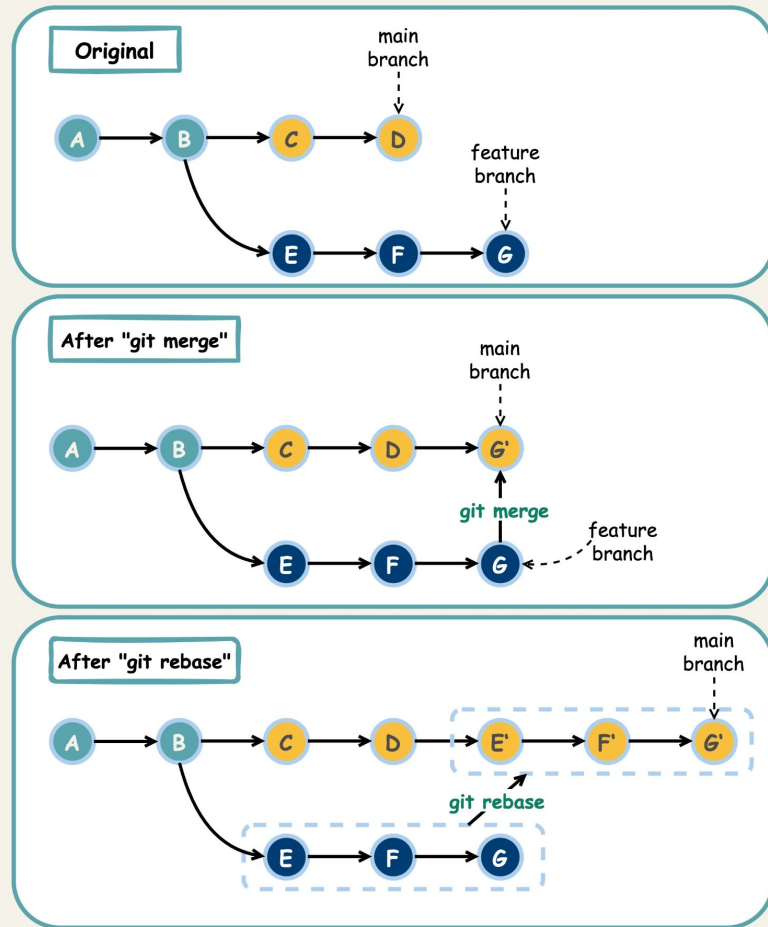
You may instead choose to incorporate newBranch into main by doing a rebase. This is an example of what a rebase may look like in your code (using the same main as previous)

```
hello.cpp > main()
1  #include <iostream>
2
3  int main()
4  {
5      "Click to add a breakpoint" "This is just a sample program." << std::endl;
6      std::cout << "Hello everybody!" << std::endl;
7      std::cout << "I am making a scary line! D:" << std::endl;
8      return EXIT_SUCCESS;
9  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

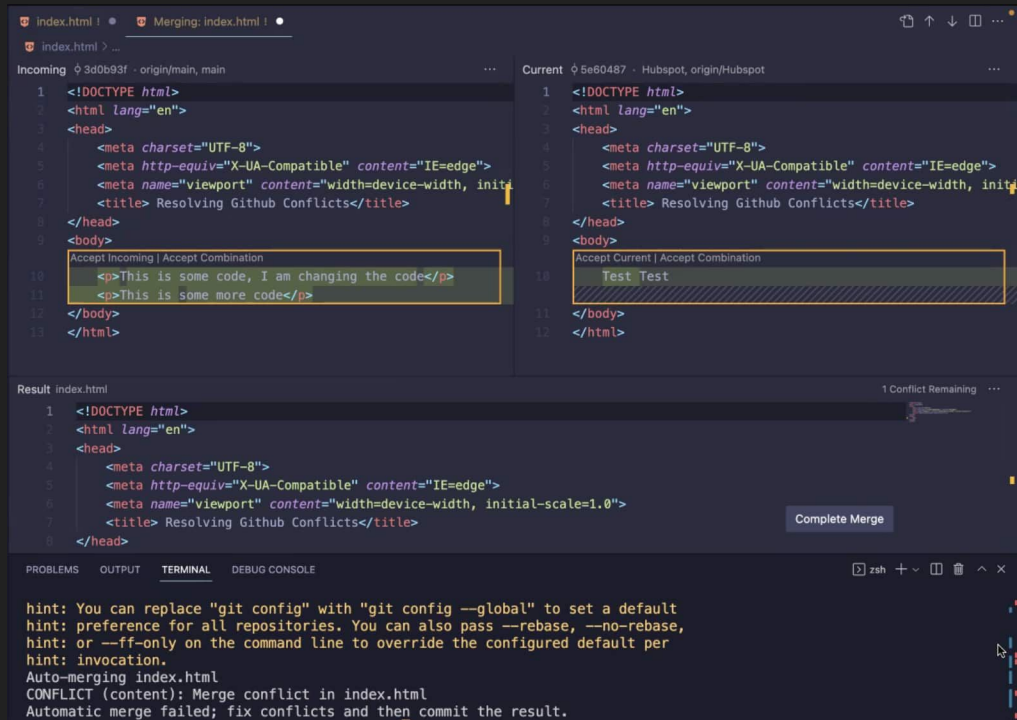
```
jay@Jays-MacBook-Air ~/.../GitHubWorkshop [main] git rebase newBranch
Successfully rebased and updated refs/heads/main.
```

Merge + Rebase



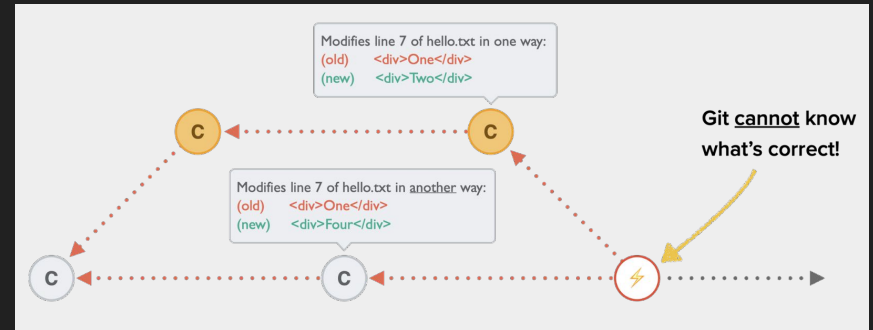
What if you get a merge conflict?

A merge conflict occurs whenever lines in a file in one branch interfere with the same lines in the merge/rebased branch. It is up to whoever is resolving the conflict to decide which version to keep, and which version to toss.



What causes a merge conflict? (example)

1. Developer A is working on a feature in file db.java.
2. Developer B is working on a different feature in the exact same file.
3. Developer A finishes their feature first, and creates a PR (Pull Request)
4. Developer A merges their PR into the main branch **before** developer B
5. Because Developer A and Developer B changed the same lines in the file with different content, a merge conflict arises
6. Developer B **must** resolve this merge conflict in order to merge their feature into main

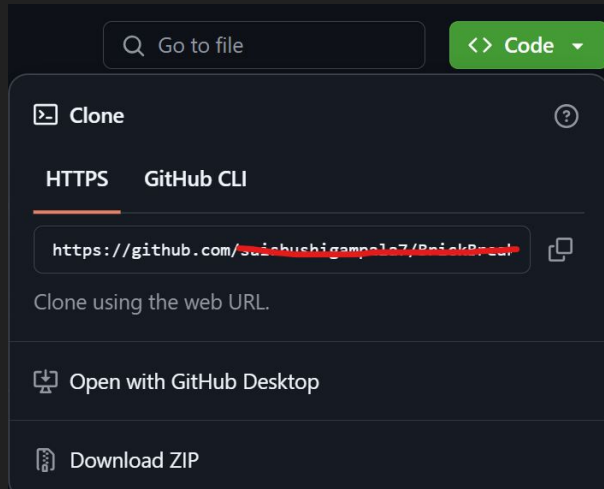


Collaboration on Github

Cloning vs. Forking

Cloning - creates a copy of the repository on your local machine

- Cloning automatically sets this as the primary remote



Forking - copies repository into your account.

- History does not automatically stay in sync with original repository
- Forking allows you to maintain the project yourself and make changes that you want to make
- If you want to contribute changes to the original, you'd have to create a pull request

